

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting
(BL 3)

Assessment Tool Weightage: 10%

QUESTIONS: 03

Total Marks:25

Annexure A

MOCK INTERVIEW QUESTIONS

Code of Conduct:

*I **Jeevan Raaja S (192421005)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	30%	20%	25%	15%	10%	
<i>Q1</i>						
<i>Q2</i>						

Signature of the student:Jeevan Raaja S

Total Marks :

Annexure A

1. You are given a small dataset containing student records with missing marks and inconsistent grade formats. How would you use **NumPy and Pandas** to clean the data, fill missing values, and standardize grades? Explain your approach in simple steps.
2. A company provides you with sales data, and you are asked to find the total and average sales for each product category. How would you use **Pandas grouping and aggregation functions** to solve this problem? Explain the steps clearly.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Cleaning Student Records Using NumPy and Pandas

Question:

You are given a small dataset containing student records with missing marks and inconsistent grade formats. How would you use NumPy and Pandas to clean the data, fill missing values, and standardize grades? Explain your approach in simple steps.

Answer

I would use **Pandas** to load and manipulate the student dataset and **NumPy** to handle numerical values and missing marks.

Step 1: Import libraries and create/load the dataset

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("students.csv")
print(df)
```

Step 2: Identify missing values

First, I would check which columns contain missing values.

```
print(df.isnull().sum())
```

This helps identify students whose marks or grades are missing.

Step 3: Fill missing marks

For numerical marks, I can replace missing values with the **mean or median** of the available marks.

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

Using the mean allows the missing mark to be replaced with a value calculated from the available student records.

Step 4: Standardize grade formats

Grades may be stored in different formats such as:

a, A, a+, A+

I would remove unnecessary spaces and convert grades to a common format.

```
df["Grade"] = df["Grade"].str.strip().str.upper()
```

For example:

"a" → "A"

" a " → "A"

"a+" → "A+"

Step 5: Use NumPy for numerical validation

NumPy can be used to ensure marks are within the valid range.

```
df["Marks"] = np.clip(df["Marks"], 0, 100)
```

This ensures that marks below 0 become 0 and marks above 100 become 100.

Step 6: Verify the cleaned data

Finally, I would check whether missing values remain and inspect the standardized grades.

```
print(df.isnull().sum())  
print(df["Grade"].unique())  
print(df)
```

If the missing-value count is 0 and grades have a consistent format, the dataset is ready for further analysis.

Simple Workflow

Load Data → Check Missing Values → Fill Missing Marks → Standardize Grades → Validate Marks → Verify Cleaned Dataset

Conclusion

Pandas provides functions such as `isnull()`, `fillna()`, and string operations for cleaning the student records, while NumPy can be used for numerical processing and validation. This approach produces a **complete, consistent, and analysis-ready student dataset**.

Q2. Finding Total and Average Sales by Product Category

Question:

A company provides you with sales data, and you are asked to find the total and average sales for each product category. How would you use Pandas grouping and aggregation functions to solve this problem? Explain the steps clearly.

Answer

I would use the Pandas **groupby()** function to group sales records according to product category and then use **aggregation functions** such as **sum()** and **mean()** to calculate the total and average sales.

Step 1: Load the sales data

```
import pandas as pd
```

```
df = pd.read_csv("sales.csv")
```

```
print(df.head())
```

Suppose the data looks like:

Product	Category	Sales
Laptop	Electronics	50000
Phone	Electronics	30000
Chair	Furniture	10000
Table	Furniture	20000

Book Stationery 5000

Step 2: Group the data by category

```
grouped = df.groupby("Category")
```

This creates separate groups for:

- Electronics
- Furniture
- Stationery

Step 3: Calculate total sales

Use the `sum()` aggregation function.

```
total_sales = df.groupby("Category")["Sales"].sum()
```

```
print(total_sales)
```

Step 4: Calculate average sales

Use the `mean()` aggregation function.

```
average_sales = df.groupby("Category")["Sales"].mean()
```

```
print(average_sales)
```

Step 5: Calculate both together

A more efficient approach is to calculate both metrics in a single operation.

```
summary = df.groupby("Category")["Sales"].agg(
```

```
    Total_Sales="sum",
```

```
    Average_Sales="mean"
```

```
)
```

```
print(summary)
```

Expected Result

Category	Total Sales	Average Sales
Electronics	80000	40000
Furniture	30000	15000
Stationery	5000	5000

Step 6: Validate the result

I would verify that the total of the category-wise sales matches the total sales in the original dataset.

```
print("Original Total:", df["Sales"].sum())
```

```
print("Grouped Total:", summary["Total_Sales"].sum())
```

If both totals are equal, the aggregation is correct.

Simple Workflow

Load Sales Data → Group by Category → Calculate Sum → Calculate Mean → Create Summary → Validate Results

Conclusion

Pandas `groupby()` divides the sales data into product categories, while `sum()` calculates the **total sales** and `mean()` calculates the **average sales** for each category. Using `agg()` allows both calculations to be performed together, producing a clear summary report for business analysis.